

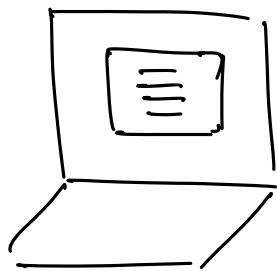
Agenda.

- Git Commands Demo.
- Git Branches.
- Merge
- Rebase
- Merge vs Rebase.
- HEAD.

⇒ GIT.

→ Distributed VCS.

→ Allows multiple developers to work on a project independently without interference.



I) git init

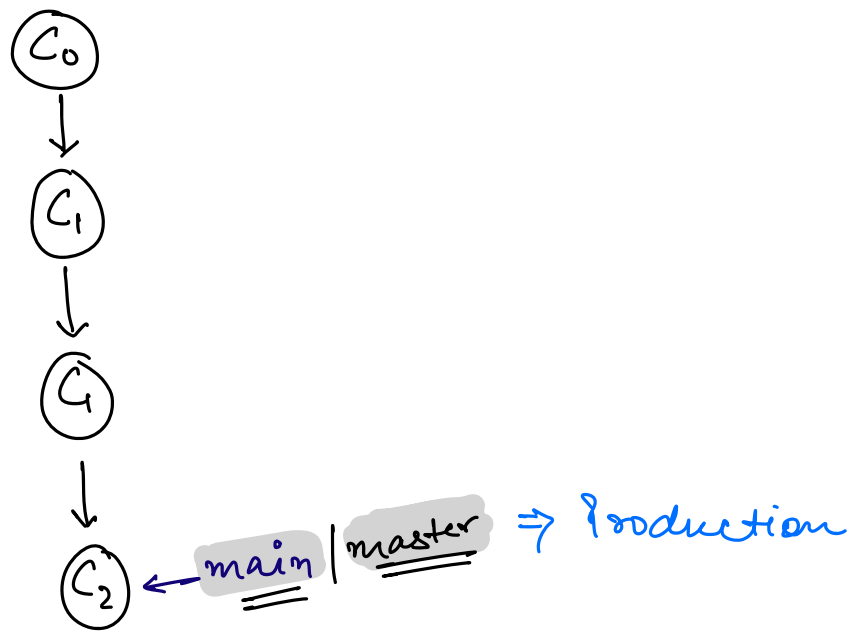
II) git add.

III) git commit -m " "

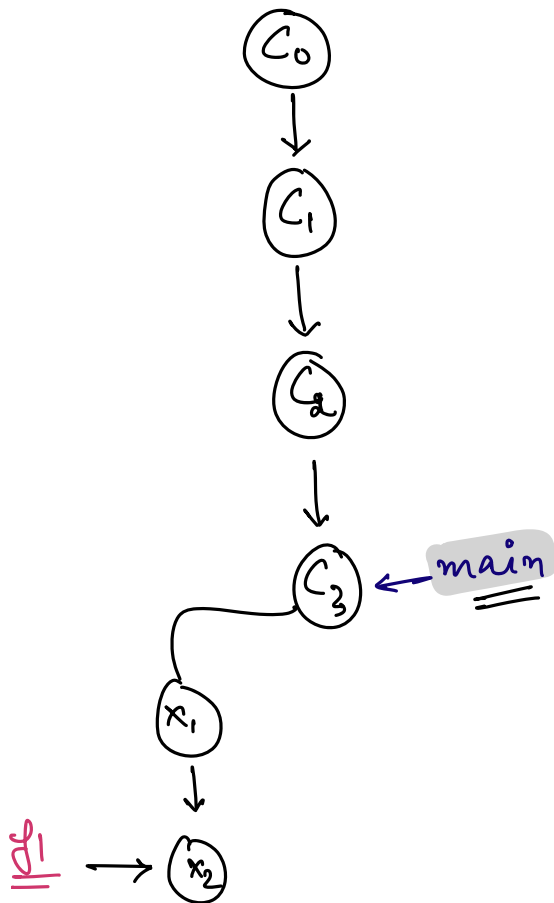
Push this
repository

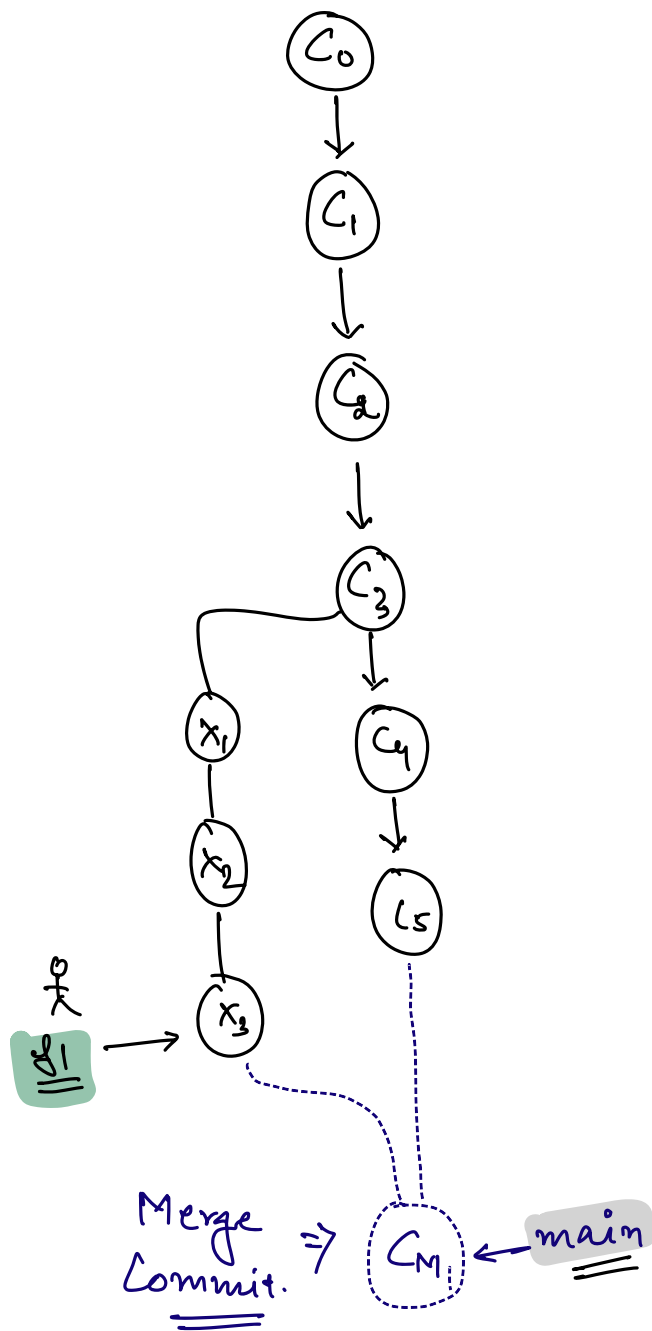
GITHUB | BITBUCKET | GITLAB.

⇒ Git Repository :- Series of Commits.



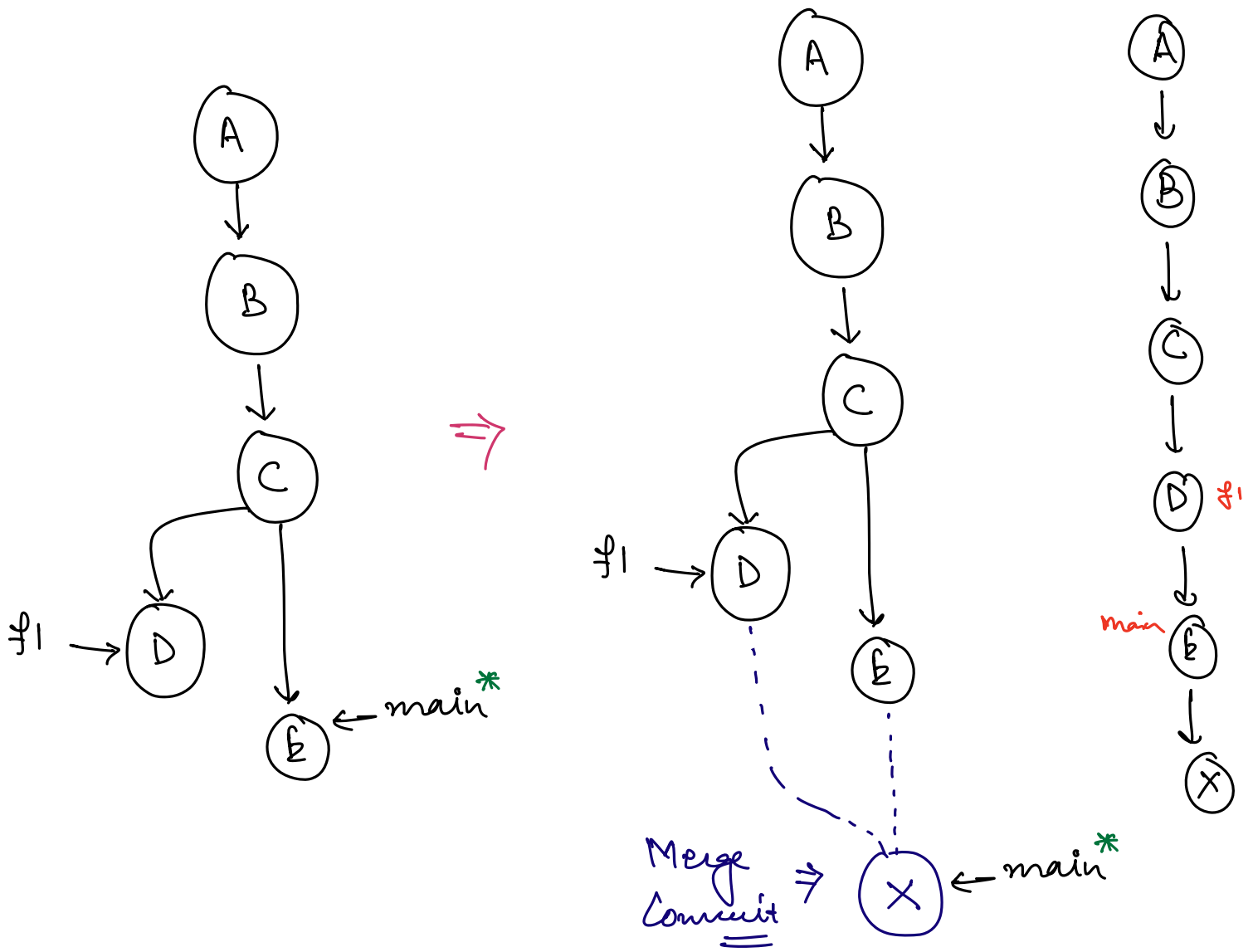
Branch: Pointer pointing to the latest commit of that branch.





⇒ Bring changes of X1 into main branch.

Git Merge : Command to bring changes of one branch into another branch.

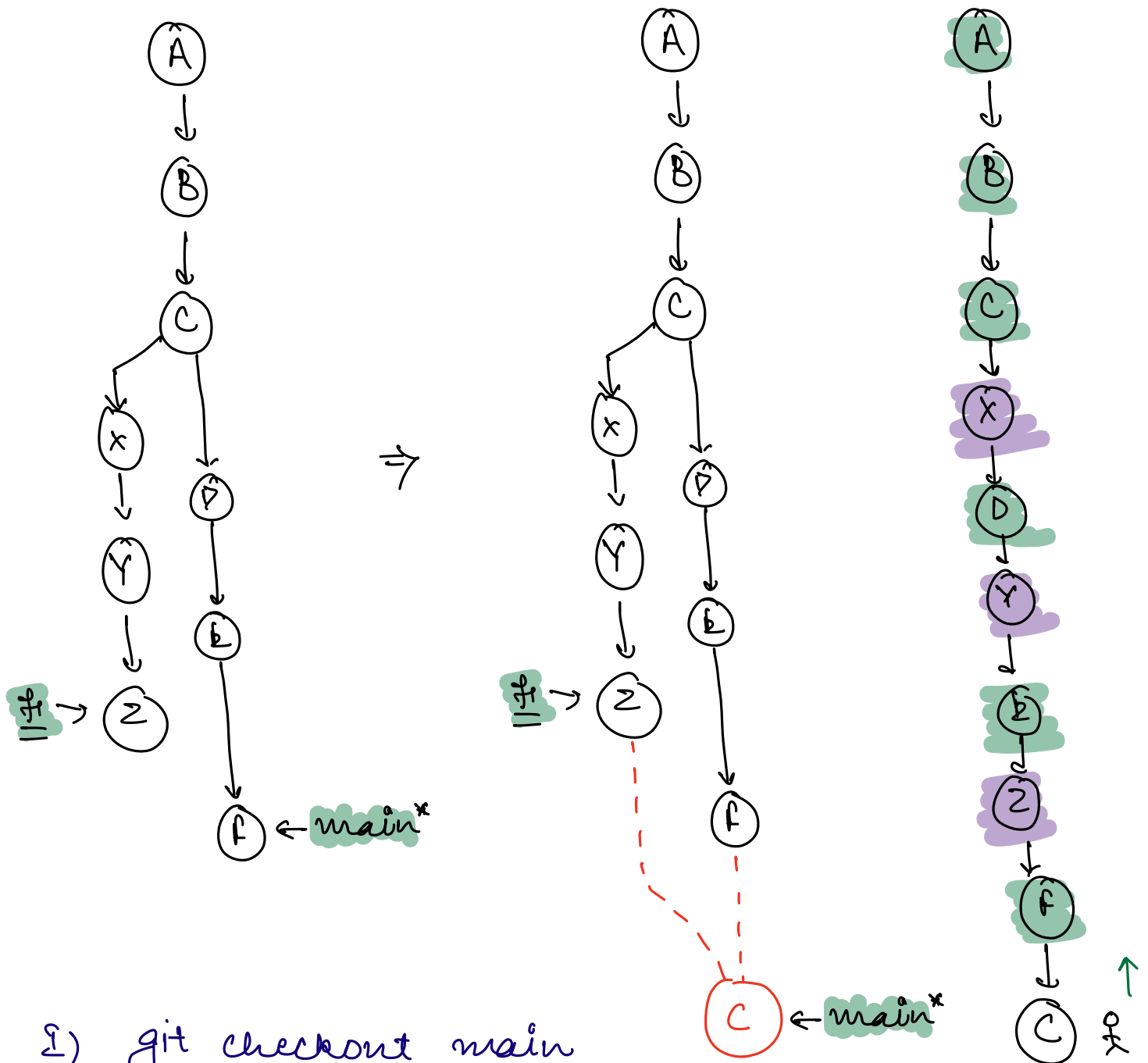


I) `git checkout main`

II) `git merge #1`

III) `git log` ⇒

X
E
D
C
B
A



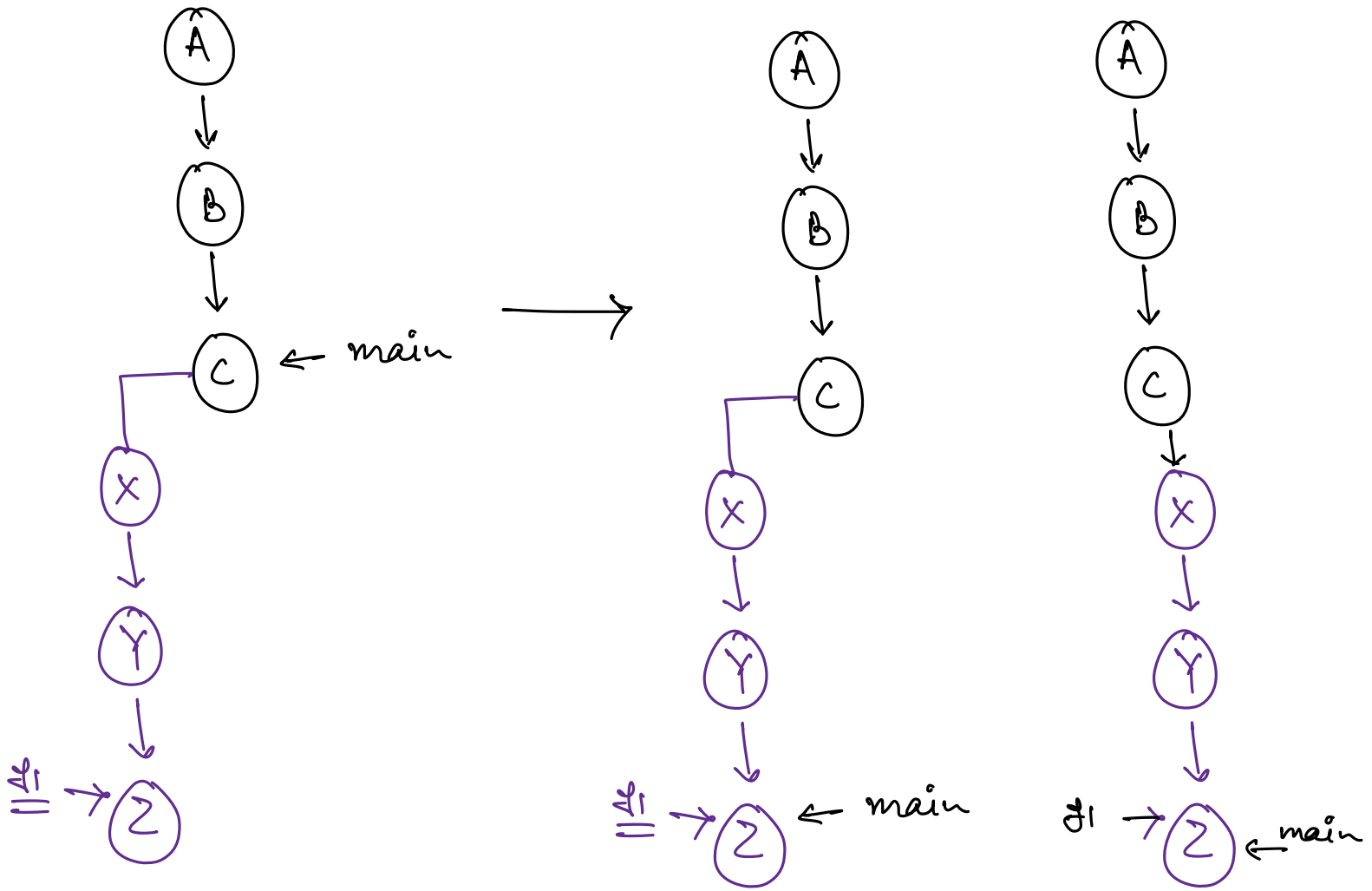
I) git checkout main

II) git merge x

III) git log

⇒ Merge is not preferred because we don't get clean history.

⇒ Merge #1 into main



⇒ Fast forward Merge.

⇒ No merge commit required.

⇒ Clean.

A.txt

1
2
3
4
5
6

Merge Conflict

#

1 1
2
3
4
5
6

main

10
2
3
4
5
6

Merge

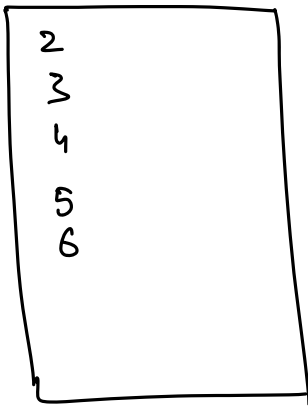
11
2
3
4
5
6

10
2
3
4
5
6

11
10
2
3
4
5
6

10
11
2
3
4
5
6

- - -

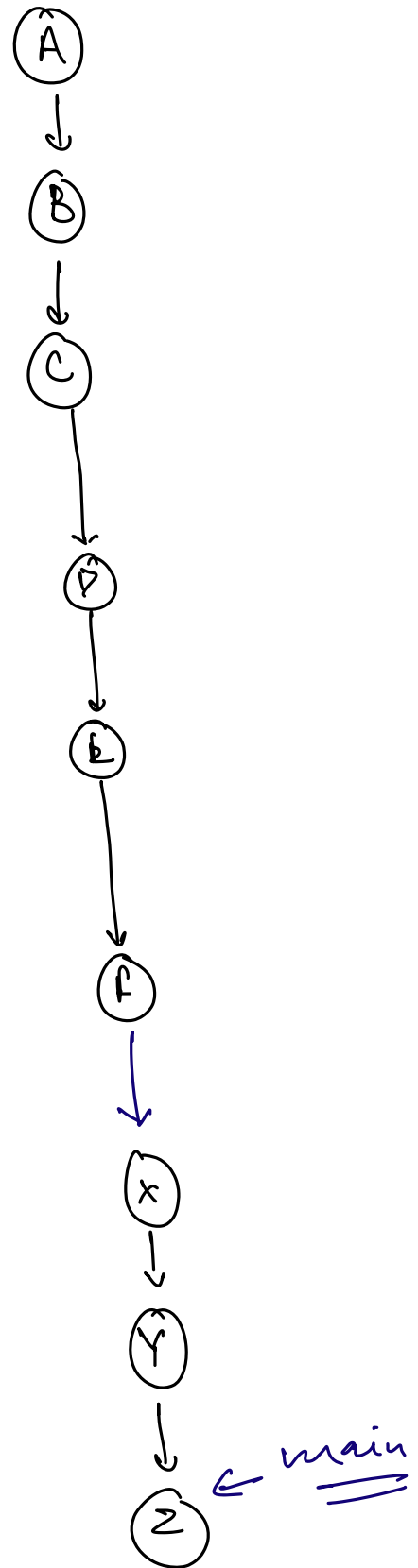
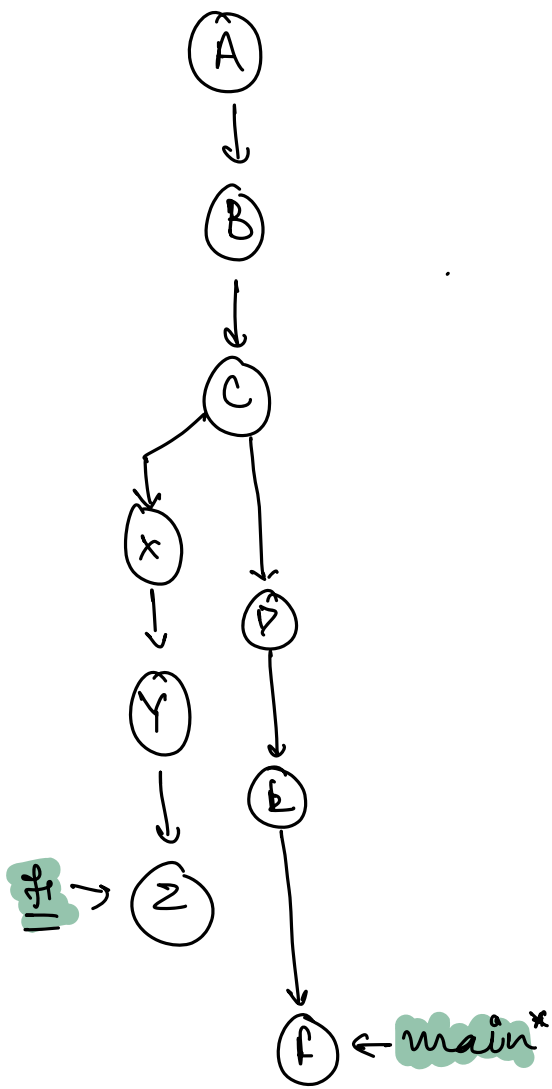


Merge Conflict.

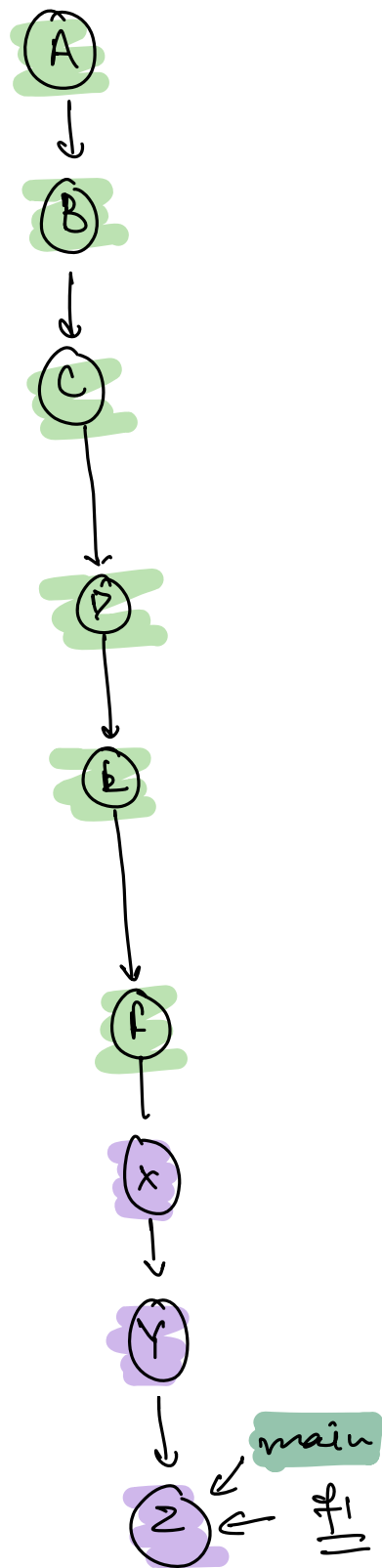
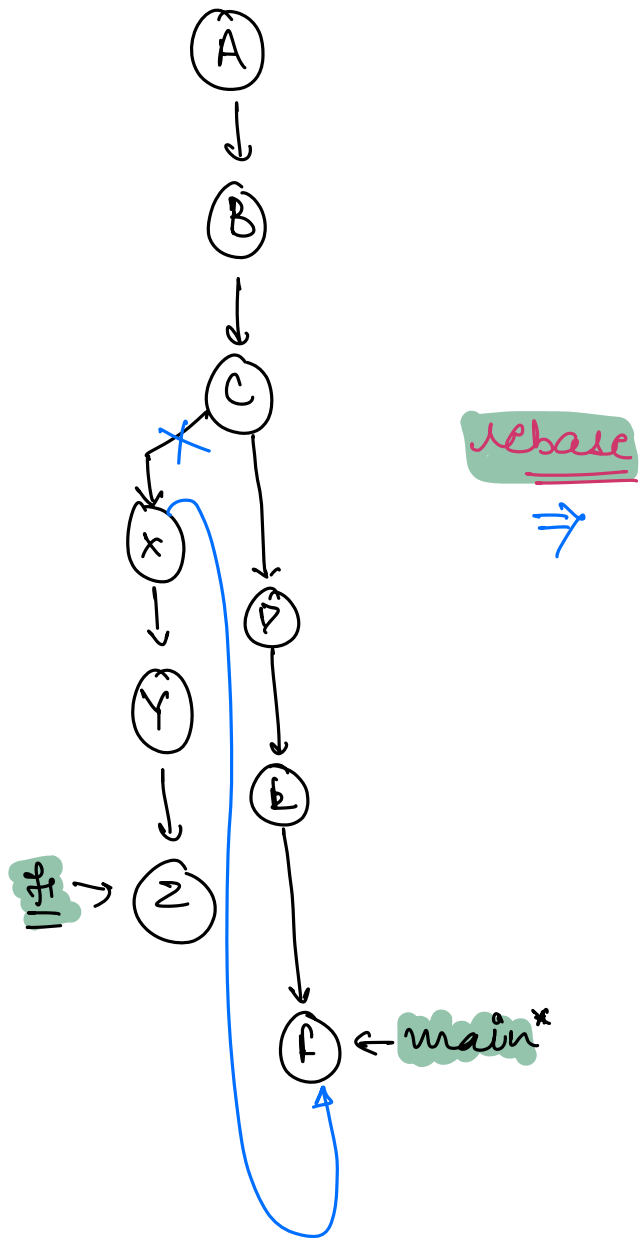
→ Manually resolved by developer.

⇒ Merge is not preferred because we don't get clean history.

→ Ideally all the changes of a branch should be present together while merging. ⇒



⇒ Rebase



Merge

≡

Rebase + Merge

↑
Merge Commit

⇒ Overlapping
history,

↓
No Merge Commit.

⇒ Cleaner History.